Pre-lab 4 and Lab 4 - DNS and TCP

Total Points: 100 (5 points for progress check in lab)

Pre-lab 4: [5 pts] Complete before you come to lab

Before getting started: Read Section 2.4: Domain Name Service (DNS) in the textbook before beginning this assignment.

Thinking about DNS:

a. Think about the role of the **Resource Records** (RR) in the **DNS** protocol and the role of a **web object** in the **HTTP** protocol. What **similarities** do they share in their respective protocols?
Both RRs in DNS and web objects in HTTP play essential roles in facilitating client-server interactions by providing necessary information for clients to access specific resources. In DNS, RRs enable clients to resolve domain names to IP addresses, while in HTTP, web objects are the resources clients request once they know the server's address.

b. Why are domain **aliases frequently** used on the Internet?
Domain aliases, also known as CNAME records, allow multiple domain names to point to the same IP address or resource. They are used for load balancing, ease of website management, and handling traffic across multiple domains. Aliases also allow a single server or IP address to host multiple services under different domain names, making it easier to manage and scale web services.

c. What is an Authoritative name server and why is it **needed**?
An Authoritative name server is a DNS server that provides the actual records of a domain and can answer with authority, providing clients with the most accurate information about a domain.

Authoritative name servers are crucial because they ensure that when users look up domain information, they receive accurate and current data directly from the source (the domain's designated DNS server), which supports the reliability and correctness of the DNS infrastructure.

d. What are root name servers? Why are they replicated and distributed around the globe?

Root name servers are the top-level DNS servers that handle requests for information about the root zone and direct queries to the appropriate top-level domain (TLD) servers.

Root servers are distributed worldwide to increase DNS resilience, reduce latency, and handle high volumes of traffic. Replication helps to ensure that even if some servers go down, others remain available.

e. What information does an **MX** resource record provide?

An MX (Mail Exchanger) record specifies the mail servers responsible for receiving email for a domain. It includes the hostname of the mail server and a priority value, which tells email servers which mail server to contact first if there are multiple options. MX records are essential for routing email properly within the DNS system.

Lab 4 - DNS and TCP

Platform: This entire lab is to be run on the Mininet VM. The lab computers are available for all of these problems.

Submission:

One PDF file for this assignment is to be submitted directly on Gradescope. Naming convention of the file: lab4_ [YourCruzID].pdf.

References: Chapter 2, Computer Networking: A Top Down Approach

- Section 2.2: The Web and HTTP
- Section 2.4: Domain Name Service (DNS)
- Chapter 3: Transport Layer
- RFC 2616: Hypertext Transfer Protocol -- HTTP/1.1
- https://www.w3.org/Protocols/rfc2616/rfc2616-sec5.html
- https://www.ietf.org/rfc/rfc1035.txt

Screenshot Check-off: (just for you to keep track of your progress)

Problem #		
Q1 a, b	Q6 b	
Q2	Q7b	
Q3a	Q8 - 1 for Query - 1 for Response	
Q4a	Q10 b, d, f	
Q5b		

[40 pts] Domain Name Service

In this lab, we'll make extensive use of the command line tool ***dig***. In its most basic operation, the `dig` tool allows the host running the tool to query any Nameserver for a DNS record. The queried DNS server can be a root DNS server, a top-level-domain DNS server, an authoritative DNS server, or an intermediate DNS server (see the textbook for definitions of these terms).

Example commands (Read man page for more information):

```
dig [domain name]  [record type:optional]
dig www.google.com
dig www.google.com AAAA
```

DNS Warmups - Resource Records:

Q1 [6 pts] Running dig :

Take a **screenshot** and highlight your answers **for (a) and (b)**

```
mininet@mininet-vm:~$ dig www.santacruz.org

; <<>> DiG 9.16.1-Ubuntu <<>> www.santacruz.org
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 47577
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;www.santacruz.org.             IN      A

;; ANSWER SECTION:
www.santacruz.org.      3600    IN      A       162.159.135.42

;; Query time: 4 msec
;; SERVER: 127.0.0.53#53(127.0.0.53)
;; WHEN: Sat Oct 26 14:05:23 PDT 2024
;; MSG SIZE  rcvd: 62
```

a. Run the command 'dig www.santacruz.org'. What **type** of Resource Record(s) are returned and **what is their function**?
An A record is received.
A record is to map a domain name to an IPv4 Address. In this case, www.santacruz.org is mapped to the IP address 162.159.135.42.

b. What is the **TTL** value of the returned Resource Record(s)?
What is the purpose of the **Time to Live (TTL)** field?
The TTL (Time to Live) value for the A record is 3600 seconds.
The TTL field indicates how long a DNS record should be cached by DNS resolvers or clients.

c. Treasure hunt! Use dig to run a reverse DNS Lookup. What command would you use to run a reverse DNS lookup on IP Address 171.67.215.200? What domain is associated with the address 171.67.215.200?
$ dig -x 171.67.215.200

web.stanford.edu is associated with the given IP address.

Q2 [4 pts] CNAME records

Run a command to find the **CNAME** record for **www.amazon.com**. Which name is the ***alias***? Which name is the **canonical** name?

$ dig www.amazon.com CNAME

Alias (CNAME) is www.amazon.com

Canonical Name is tp.47cf2c8c9-frontier.amazon.com

Take a **screenshot** of the command and output – highlight the **alias** and **canonical** name to support your answer.

```
mininet@mininet-vm:~$ dig www.amazon.com CNAME

; <<>> DiG 9.16.1-Ubuntu <<>> www.amazon.com CNAME
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 21166
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;www.amazon.com.                IN      CNAME

;; ANSWER SECTION:
www.amazon.com.         806     IN      CNAME   tp.47cf2c8c9-frontier.amazon.com.

;; Query time: 3 msec
;; SERVER: 127.0.0.53#53(127.0.0.53)
;; WHEN: Sat Oct 26 14:12:45 PDT 2024
;; MSG SIZE  rcvd: 79
```

Q3 [6 pts] Run **dig** to find the ***MX*** resource record of **gmail.com**

a. What **command** did you use to obtain the **MX** resource records for the given domain? Take a **screenshot** of the command and output – to support your answer.

$ dig gmail.com MX

```
mininet@mininet-vm:~$ dig gmail.com MX

; <<>> DiG 9.16.1-Ubuntu <<>> gmail.com MX
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 3965
;; flags: qr rd ra; QUERY: 1, ANSWER: 5, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;gmail.com.                     IN      MX

;; ANSWER SECTION:
gmail.com.              201     IN      MX      40 alt4.gmail-smtp-in.l.google.com.
gmail.com.              201     IN      MX      20 alt2.gmail-smtp-in.l.google.com.
gmail.com.              201     IN      MX      30 alt3.gmail-smtp-in.l.google.com.
gmail.com.              201     IN      MX      10 alt1.gmail-smtp-in.l.google.com.
gmail.com.              201     IN      MX      5 gmail-smtp-in.l.google.com.

;; Query time: 4 msec
;; SERVER: 127.0.0.53#53(127.0.0.53)
;; WHEN: Sat Oct 26 14:15:10 PDT 2024
;; MSG SIZE  rcvd: 161

mininet@mininet-vm:~$
```

b. Think about accessing this domain in your browser compared to sending an email to a person with an email account in the domain (e.g., www.ucsc.edu in your browser versus sammyslug@ucsc.edu).
What corresponding **DNS** queries are made, how are they **different**?
What **DNS** mechanisms/services are being used in this example?
When accessing a domain in a browser, A record (Address record) query is made. The DNS server returns an IP address that maps to the domain (e.g., www.ucsc.edu) so that the browser knows where to connect to retrieve the website.

When sending an email, an MX record query is made. The DNS server returns one or more mail servers responsible for receiving emails for the domain (e.g., ucsc.edu). Each MX record includes a priority value, allowing the sending mail server to select which server to contact first.

For this example, an MX record is being used because mail servers are associated with the email domain.

Q4 [4 pts] Authoritative name servers

a. Run **`dig`** to determine the authoritative **DNS** servers for a university in **North America.** What is the name of the university you chose?
Take a **screenshot** highlighting the output showing the Authoritative NS. What is it in the screenshot that shows you have found this type of nameserver.
I chose Stanford University. I ran $ dig stanford.edu NS

The authoritative name servers for stanford.edu are

1. argus.stanford.edu
2. atalante.stanford.edu
3. avallone.stanford.edu
4. ns7.dnsmadeeasy.com
5. ns6.dnsmadeeasy.com
6. ns5.dnsmadeeasy.com

```
mininet@mininet-vm:~$ dig stanford.edu NS

; <<>> DiG 9.16.1-Ubuntu <<>> stanford.edu NS
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 48929
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;stanford.edu.                  IN      NS

;; ANSWER SECTION:
stanford.edu.           86400   IN      NS      argus.stanford.edu.
stanford.edu.           86400   IN      NS      atalante.stanford.edu.
stanford.edu.           86400   IN      NS      ns7.dnsmadeeasy.com.
stanford.edu.           86400   IN      NS      ns6.dnsmadeeasy.com.
stanford.edu.           86400   IN      NS      ns5.dnsmadeeasy.com.
stanford.edu.           86400   IN      NS      avallone.stanford.edu.

;; Query time: 7 msec
;; SERVER: 127.0.0.53#53(127.0.0.53)
;; WHEN: Sat Oct 26 14:20:20 PDT 2024
;; MSG SIZE  rcvd: 176
```

a. Suppose the university wanted to further partition the **DNS** namespace. For example, the **econ** and **computer science** departments want to be in charge of their own **namespace** (e.g.,names of computers in their respective departments). How could **Authoritative nameservers** be used to accomplish this task?

Stanford's DNS administrators could create subdomains such as econ.stanford.edu for the economics department and cs.stanford.edu for the computer science department. Another way could be setting up a departmental authoritative name server responsible for its subdomain. For example, ns1.econ.stanford.edu for the economics department.

Q5 [6 pts] Run `dig amazon.com ANY` and attach a screenshot.

Note: When running the dig ANY command, you should be on UCSC campus

```
mininet@mininet-vm:~$ dig amazon.com ANY

; <<>> DiG 9.16.1-Ubuntu <<>> amazon.com ANY
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 63043
;; flags: qr rd ra; QUERY: 1, ANSWER: 12, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;amazon.com.                    IN      ANY

;; ANSWER SECTION:
amazon.com.             3600    IN      HINFO   "RFC8482" ""
amazon.com.             10      IN      A       205.251.242.103
amazon.com.             10      IN      A       54.239.28.85
amazon.com.             10      IN      A       52.94.236.248
amazon.com.             17750   IN      NS      ns1.amzndns.net.
amazon.com.             17750   IN      NS      ns1.amzndns.com.
amazon.com.             17750   IN      NS      ns2.amzndns.co.uk.
amazon.com.             17750   IN      NS      ns2.amzndns.com.
amazon.com.             17750   IN      NS      ns2.amzndns.net.
amazon.com.             17750   IN      NS      ns1.amzndns.org.
amazon.com.             17750   IN      NS      ns1.amzndns.co.uk.
amazon.com.             17750   IN      NS      ns2.amzndns.org.

;; Query time: 7 msec
;; SERVER: 127.0.0.53#53(127.0.0.53)
;; WHEN: Sat Oct 26 14:25:15 PDT 2024
;; MSG SIZE  rcvd: 295
```

a) What is the purpose of ANY?

ANY query type in DNS requests all types of resource records associated with a domain name. This can include A (Address), NS (Name Server), MX (Mail Exchanger), TXT (Text), and other records, depending on what the DNS server has for that domain.

b) What do the multiple A records represent? Add a **screenshot** and highlight the multiple A records being returned by the dig command.

The multiple A records represent different IP addresses that amazon.com resolves to. Multiple A records are often used for load balancing and redundancy.

The IP addresses listed in the A record are:

- 205.251.242.103
- 54.239.28.85
- 52.94.236.248

Q6 [4 pts] Using a Clean Cache

Open Wireshark and listen on the `'any'` interface. Open a terminal, and clear your dns cache using the command `systemd-resolve --flush-caches`

Then, use `dig` to perform a DNS lookup to x.com **Two consecutive times**.

a. Comparing the two runs of dig, is there a change in the query time?
   Yes. There was a change in query time.
   The first run of dig, the query took 3 msec, while the second query took 1 msec.

b. With respect to what you know about DNS, provide an explanation for the difference in lookup time - i.e. what is different between the two queries? Include a **screenshot** and refer to the results in your explanation.
   The difference in lookup times between the two queries is due to caching.
   - First Query: The resolver doesn't have the IP address for x.com cached, so it has to perform a full DNS resolution process, which takes more time.
   - Second Query: After the first query, the IP address for x.com is stored in the DNS cache. In the second query, the resolver can quickly return the result from the cache without needing to communicate with external DNS servers, resulting in a faster response time.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	UDP	45	44045 → 44045 Len=1
2	0.001120316	127.0.0.1	127.0.0.53	DNS	90	Standard query 0x4283 A x.com OPT
3	0.001285400	10.0.2.15	10.0.2.3	DNS	78	Standard query 0xa8ce A x.com OPT
4	0.006007932	10.0.2.3	10.0.2.15	DNS	94	Standard query response 0xa8ce A x.com A 104.244.42.193 OPT
5	0.006141593	127.0.0.53	127.0.0.1	DNS	94	Standard query response 0x4283 A x.com A 104.244.42.193 OPT
6	2.343884696	127.0.0.1	127.0.0.1	UDP	45	47347 → 47347 Len=1
7	2.344997388	127.0.0.1	127.0.0.53	DNS	90	Standard query 0x2244 A x.com OPT
8	2.345099484	127.0.0.53	127.0.0.1	DNS	94	Standard query response 0x2244 A x.com A 104.244.42.193 OPT
9	5.230097934	PcsCompu_90:2b:13		ARP	44	Who has 10.0.2.3? Tell 10.0.2.15
10	5.230790035	52:55:0a:00:02:03		ARP	66	10.0.2.3 is at 52:55:0a:00:02:03
11	6.836398731	10.0.2.15	198.137.202.56	NTP	92	NTP Version 4, client
12	6.839120561	198.137.202.56	10.0.2.15	NTP	92	NTP Version 4, server

```
mininet@mininet-vm:~$ dig x.com

; <<>> DiG 9.16.1-Ubuntu <<>> x.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 17027
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;x.com.                         IN      A

;; ANSWER SECTION:
x.com.                  311     IN      A       104.244.42.193

;; Query time: 3 msec
;; SERVER: 127.0.0.53#53(127.0.0.53)
;; WHEN: Sat Oct 26 14:30:07 PDT 2024
;; MSG SIZE  rcvd: 50

mininet@mininet-vm:~$ dig x.com

; <<>> DiG 9.16.1-Ubuntu <<>> x.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 8772
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;x.com.                         IN      A

;; ANSWER SECTION:
x.com.                  308     IN      A       104.244.42.193

;; Query time: 0 msec
;; SERVER: 127.0.0.53#53(127.0.0.53)
;; WHEN: Sat Oct 26 14:30:09 PDT 2024
;; MSG SIZE  rcvd: 50
```

The initial query (packets 5-6) takes around 2.34 seconds (according to the wireshark timing)

The second query takes around 0.001 seconds to complete (packet 7 was sent at 2.344 and packet 8 was received at 2.345)

The two dig runs show a difference in query times. The first one is much slower than the second one.

Q7 [10 pts] Root Name Servers

a. What are 2 different commands that can be used to find the root name servers?
   $ dig . NS
   $ nslookup -type=ns .

b. Run one of the commands and include a **screenshot of** the results and discuss what you see.

```
mininet@mininet-vm:~$ dig . NS

; <<>> DiG 9.16.1-Ubuntu <<>> . NS
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 52891
;; flags: qr rd ra; QUERY: 1, ANSWER: 13, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;.                              IN      NS

;; ANSWER SECTION:
.                       400861  IN      NS      b.root-servers.net.
.                       400861  IN      NS      h.root-servers.net.
.                       400861  IN      NS      e.root-servers.net.
.                       400861  IN      NS      k.root-servers.net.
.                       400861  IN      NS      d.root-servers.net.
.                       400861  IN      NS      c.root-servers.net.
.                       400861  IN      NS      m.root-servers.net.
.                       400861  IN      NS      i.root-servers.net.
.                       400861  IN      NS      g.root-servers.net.
.                       400861  IN      NS      f.root-servers.net.
.                       400861  IN      NS      a.root-servers.net.
.                       400861  IN      NS      l.root-servers.net.
.                       400861  IN      NS      j.root-servers.net.

;; Query time: 7 msec
;; SERVER: 127.0.0.53#53(127.0.0.53)
;; WHEN: Sat Oct 26 14:45:41 PDT 2024
;; MSG SIZE  rcvd: 239
```

   The answer section shows a list of 13 root name servers, such as b.root-servers.net, h.root-servers.net, and so on. The query time was 7 msec and the server's IP address is 127.0.0.53

c. Is your access network physically close to a root name server? Explain how you have come to your conclusion.
   The query time is 7 msec, which is relatively fast, suggesting that the DNS resolver did not have to make extensive external requests. It also implies that the network is either physically close to an anycast instance of a root server or served by a caching DNS server that already has the root server information.

[50 pts] Transport Control Protocol - TCP

In this section we begin looking at the TCP protocol. We will give an express TCP overview in class, but we won't officially reach this topic until we study the Transport Layer in Chapter 3 (a good reference if you have questions).

Digging in with Wireshark - DNS Messages: The step before web pages are loaded

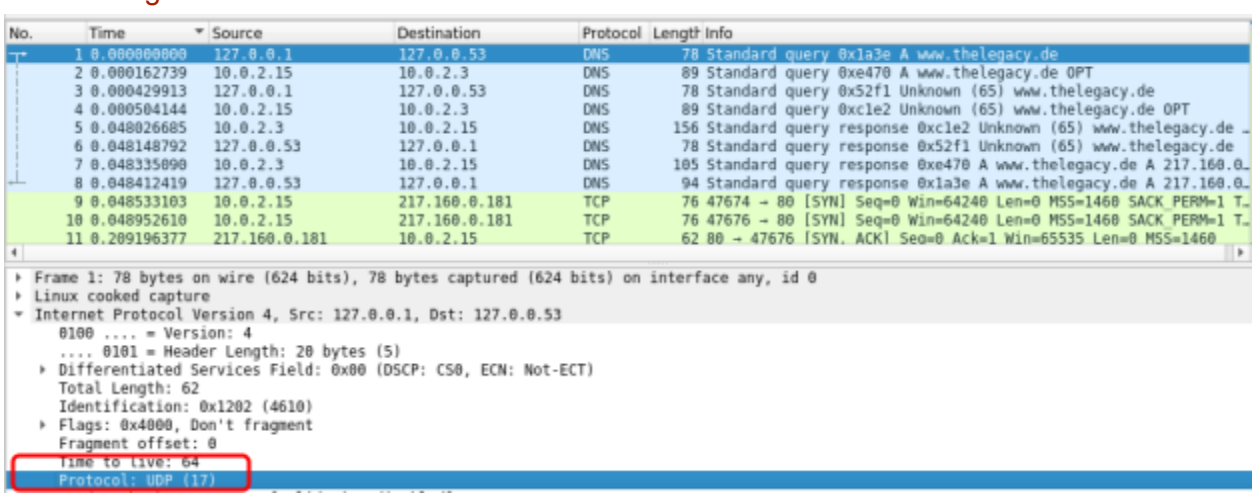
For the next two questions, we will observe how the DNS protocol operates at the packet level by using the Mininet VM and Wireshark. Begin by doing the following:

- Open Wireshark and listen on the 'any' interface without any filter
- Open Chromium, clear your web cache and navigate to
- http://www.thelegacy.de/

Q8 [15 pts] Request Web page –Wireshark capture of DNS Messages:

Show a **screenshot** in Wireshark and highlight the areas in your answers to the questions below:

a. Find the DNS query and response messages. Are these messages sent with UDP or TCP? **Circle the transport layer protocol in your screenshot.**
The messages are sent with UDP



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.53	DNS	78	Standard query 0x1a3e A www.thelegacy.de
2	0.000162739	10.0.2.15	10.0.2.3	DNS	89	Standard query 0xe470 A www.thelegacy.de OPT
3	0.000429913	127.0.0.1	127.0.0.53	DNS	78	Standard query 0x52f1 Unknown (65) www.thelegacy.de
4	0.000504144	10.0.2.15	10.0.2.3	DNS	89	Standard query 0xc1e2 Unknown (65) www.thelegacy.de OPT
5	0.048026685	10.0.2.3	10.0.2.15	DNS	156	Standard query response 0xc1e2 Unknown (65) www.thelegacy.de
6	0.048148792	127.0.0.53	127.0.0.1	DNS	78	Standard query response 0x52f1 Unknown (65) www.thelegacy.de
7	0.048335090	10.0.2.3	10.0.2.15	DNS	105	Standard query response 0xe470 A www.thelegacy.de A 217.160.0
8	0.048412419	127.0.0.53	127.0.0.1	DNS	94	Standard query response 0x1a3e A www.thelegacy.de A 217.160.0
9	0.048533103	10.0.2.15	217.160.0.181	TCP	76	47674 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 T
10	0.048952610	10.0.2.15	217.160.0.181	TCP	76	47676 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 T
11	0.209196377	217.160.0.181	10.0.2.15	TCP	62	80 → 47676 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460

```
▸ Frame 1: 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface any, id 0
▸ Linux cooked capture
▾ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.53
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 62
    Identification: 0x1202 (4610)
  ▸ Flags: 0x4000, Don't fragment
    Fragment offset: 0
    Time to Live: 64
    Protocol: UDP (17)
```

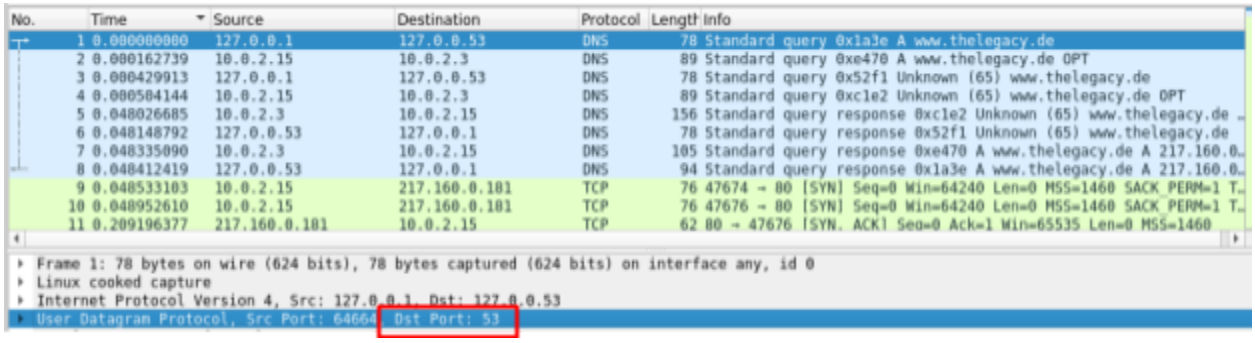
b. Discuss the transport layer protocol observed in (a) - why do you think the one you observed is used?
DNS protocol uses UDP due to its lower overhead, efficiency, and faster transmission (since no handshake is needed).

c. Look for the source and destination ports of the **DNS Queries**:

		Well known port? yes/no
Source Port Number	64664	no
Destination Port Number	53	yes

d. Is this destination port number what you expected? Why or why not? **Highlight this port in your screenshot.**

Yes. The destination port is 53 and was expected because port 53 is the well-known port designated for the DNS (Domain Name System) protocol.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.53	DNS	78	Standard query 0x1a3e A www.thelegacy.de
2	0.000162739	10.0.2.15	10.0.2.3	DNS	89	Standard query 0xe470 A www.thelegacy.de OPT
3	0.000429913	127.0.0.1	127.0.0.53	DNS	78	Standard query 0x52f1 Unknown (65) www.thelegacy.de
4	0.000504144	10.0.2.15	10.0.2.3	DNS	89	Standard query 0xc1e2 Unknown (65) www.thelegacy.de OPT
5	0.048026685	10.0.2.3	10.0.2.15	DNS	156	Standard query response 0xc1e2 Unknown (65) www.thelegacy.de
6	0.048148792	127.0.0.53	127.0.0.1	DNS	78	Standard query response 0x52f1 Unknown (65) www.thelegacy.de
7	0.048335090	10.0.2.3	10.0.2.15	DNS	105	Standard query response 0xe470 A www.thelegacy.de A 217.160.0
8	0.048412419	127.0.0.53	127.0.0.1	DNS	94	Standard query response 0x1a3e A www.thelegacy.de A 217.160.0
9	0.048533103	10.0.2.15	217.160.0.181	TCP	76	47674 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 T
10	0.048952610	10.0.2.15	217.160.0.181	TCP	76	47676 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 T
11	0.209196377	217.160.0.181	10.0.2.15	TCP	62	80 → 47676 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460

```
Frame 1: 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface any, id 0
Linux cooked capture
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.53
User Datagram Protocol, Src Port: 64664, Dst Port: 53
```

e. If you were to close your browser and then access the site again, do you expect your source port number to stay the same? Explain your answer.

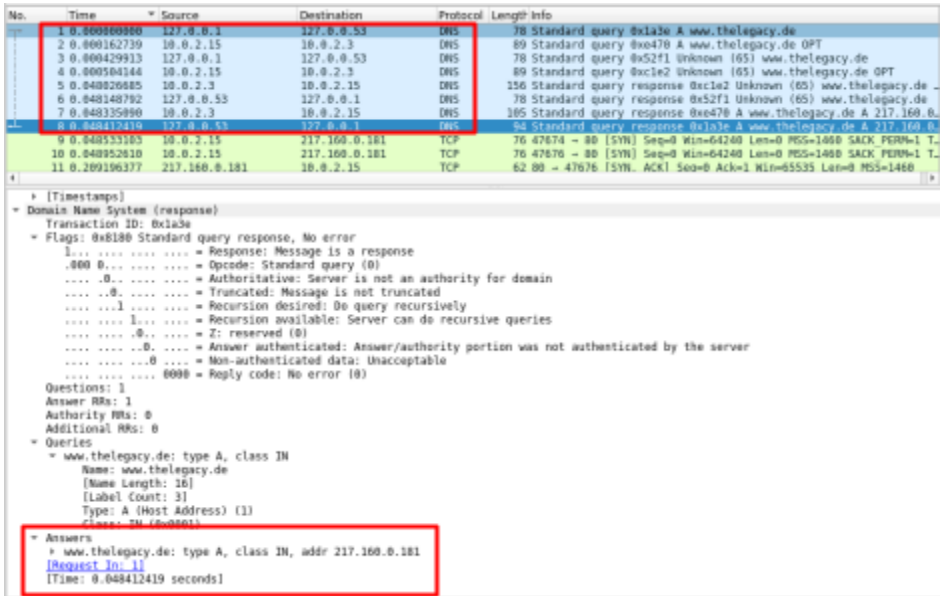
The source port would change. For outgoing DNS queries, the client typically uses an ephemeral port as the source port. Ephemeral ports are temporary and dynamically assigned by the operating system from a predefined range. Each new DNS query or session is likely to be assigned a different ephemeral port.

f. Examine the DNS response message.

- How many "answers" are provided? 8 answers
- Do you see a final answer or only referrals? If there is a final answer, what is the "final answer"? i.e. what is the IP address returned by DNS?

  Yes there is a final answer. The final IP address returned is 217.160.0.181

Highlight in screenshot and refer to the highlighted fields in your discussion.



```
[Timestamps]
Domain Name System (response)
    Transaction ID: 0x1a3e
    Flags: 0x8180 Standard query response, No error
        1... .... .... .... = Response: Message is a response
        .000 0... .... .... = Opcode: Standard query (0)
        .... .0.. .... .... = Authoritative: Server is not an authority for domain
        .... ..0. .... .... = Truncated: Message is not truncated
        .... ...1 .... .... = Recursion desired: Do query recursively
        .... .... 1... .... = Recursion available: Server can do recursive queries
        .... .... .0.. .... = Z: reserved (0)
        .... .... ..0. .... = Answer authenticated: Answer/authority portion was not authenticated by the server
        .... .... ...0 .... = Non-authenticated data: Unacceptable
        .... .... .... 0000 = Reply code: No error (0)
    Questions: 1
    Answer RRs: 1
    Authority RRs: 0
    Additional RRs: 0
    Queries
        www.thelegacy.de: type A, class IN
            Name: www.thelegacy.de
            [Name Length: 16]
            [Label Count: 3]
            Type: A (Host Address) (1)
            Class: IN (0x0001)
    Answers
        www.thelegacy.de: type A, class IN, addr 217.160.0.181
        [Request In: 1]
        [Time: 0.048412419 seconds]
```

Q9 [10 pts] Request Web page – post DNS:

This problem is a continuation of Q8

a. After DNS is finally resolved, we expect that a TCP connection should be opened <u>before</u> the HTTP request is sent. Let's verify the sequence of events.
<u>Look at a TCP packet and determine the source and destination addresses and ports of the TCP connection.</u> Fill in the table below.

IP Addresses:		
Source IP Address		10.0.2.15
Destination IP Address		217.160.0.181
Ports:		
		Well known port? yes/no
Source Port Number	47674	no
Destination Port Number	80	yes

b. Why is the destination IP address for the HTTP request different from the destination IP address used by DNS queries?
When performing a DNS query, you are reaching out to a DNS server to resolve a domain name to an IP address. The DNS server has its own IP address, which is different from the web server's IP.

Thus, the destination IP address for DNS is the IP of the DNS resolver, while the destination IP address for HTTP is the IP of the actual web server hosting the requested resource.

c. What kind of file is downloaded from the web server? How do you know?
Various different files are downloaded from the server. Some examples are the-legacy.js, the-legacy.css, JPEG, and jpg.

This can be known from the HTTP requests. The client sends GET requests for these files to the server.

Q10 [25 pts] File Transfer using **curl** from an origin server

Open Wireshark and listen on the 'any' interface. Then using **`curl`**, download http://ipv4.download.thinkbroadband.com/10MB.zip in a terminal window.

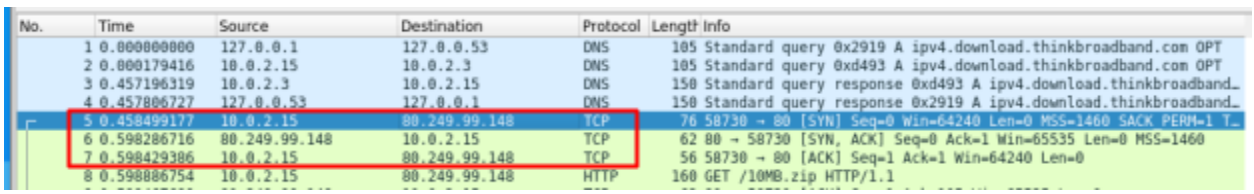
a. Enumerate and describe the usage of all of the different protocols you see in your Wireshark capture from the time you enter the URL leading up to and including the file download (Ignore NTP packets).
   4 DNS queries, 1 HTTP request, 877 TCP requests

   The DNS was used to resolve the domain name to an IP address.

   The HTTP get request is sent to download the file.

   TCP is used for file transfer (the first three are used for connection establishment). The data is broken down into TCP segments and transmitted from the server to the system.

b. Find the **TCP three-way handshake** for connection establishment. Take a **screenshot** of these TCP segments and highlight all of them in the screenshot.

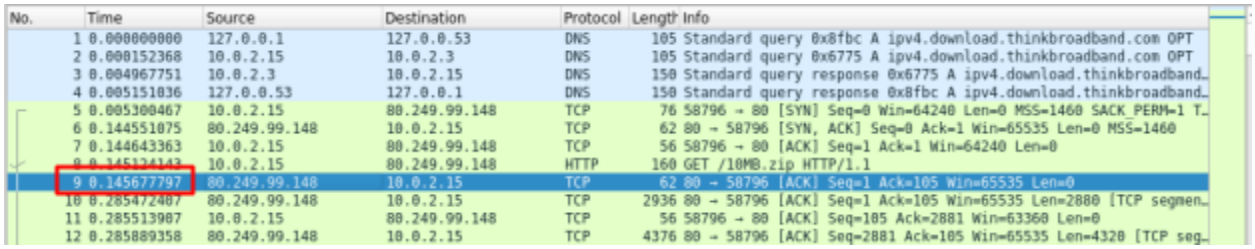


No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.53	DNS	105	Standard query 0x2919 A ipv4.download.thinkbroadband.com OPT
2	0.000179416	10.0.2.15	10.0.2.3	DNS	105	Standard query 0xd493 A ipv4.download.thinkbroadband.com OPT
3	0.457196319	10.0.2.3	10.0.2.15	DNS	150	Standard query response 0xd493 A ipv4.download.thinkbroadband…
4	0.457806727	127.0.0.53	127.0.0.1	DNS	150	Standard query response 0x2919 A ipv4.download.thinkbroadband…
5	0.458499177	10.0.2.15	80.249.99.148	TCP	76	58730 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 T…
6	0.598286716	80.249.99.148	10.0.2.15	TCP	62	80 → 58730 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
7	0.598429386	10.0.2.15	80.249.99.148	TCP	56	58730 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
8	0.598886754	10.0.2.15	80.249.99.148	HTTP	160	GET /10MB.zip HTTP/1.1

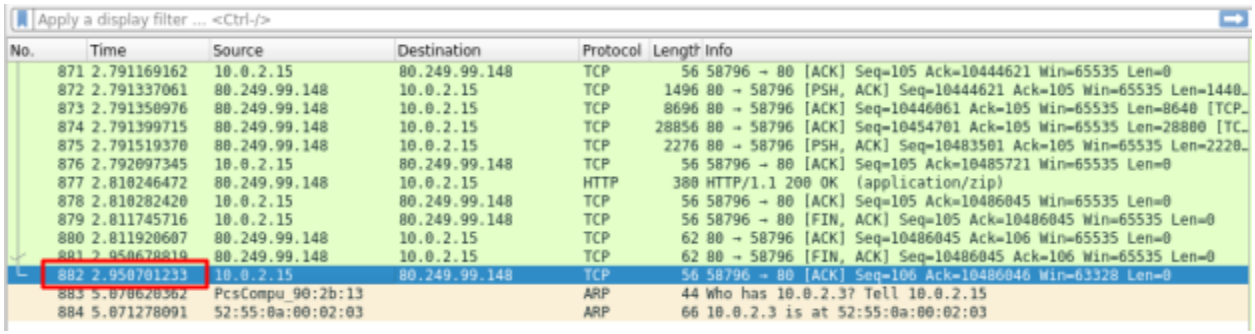
c. **Packets, packets, packets:** After the handshake is complete, observe the transfer of packets in Wireshark:
   - How many different TCP segments are transmitted to send the entire file? How did you discover this in Wireshark?
     10 segments
   - What is the typical segment size?
     Around 1200 bytes
   - What value do you expect if you multiply the number segments transmitted by the typical number of bytes in a segment? Try it! Show your work and explain.
     1200 bytes * 874 = 10.5MB

d. **File Distribution Time according to Wireshark:** Find two TCP segments (packets) – one indicating the start of the file download and then the final packet in the transfer. Using these two packets, calculate the download time using the Time column in Wireshark. Take a **screenshot** to show the two packets used in the calculation.

   The file download start packet began at time 0.14568 and the final packet was at time 2.9507. The total time for the file transfer was 2.9507 - 0.14568 = 2.80502 seconds.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.53	DNS	105	Standard query 0x8fbc A ipv4.download.thinkbroadband.com OPT
2	0.000152368	10.0.2.15	10.0.2.3	DNS	105	Standard query 0x6775 A ipv4.download.thinkbroadband.com OPT
3	0.004967751	10.0.2.3	10.0.2.15	DNS	150	Standard query response 0x6775 A ipv4.download.thinkbroadband…
4	0.005151036	127.0.0.53	127.0.0.1	DNS	150	Standard query response 0x8fbc A ipv4.download.thinkbroadband…
5	0.005300467	10.0.2.15	80.249.99.148	TCP	76	58796 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 T…
6	0.144551075	80.249.99.148	10.0.2.15	TCP	62	80 → 58796 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
7	0.144643363	10.0.2.15	80.249.99.148	TCP	56	58796 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
8	0.145124143	10.0.2.15	80.249.99.148	HTTP	160	GET /10MB.zip HTTP/1.1
9	0.145677797	80.249.99.148	10.0.2.15	TCP	62	80 → 58796 [ACK] Seq=1 Ack=105 Win=65535 Len=0
10	0.285472487	80.249.99.148	10.0.2.15	TCP	2936	80 → 58796 [ACK] Seq=1 Ack=105 Win=65535 Len=2880 [TCP segmen…
11	0.285513987	10.0.2.15	80.249.99.148	TCP	56	58796 → 80 [ACK] Seq=105 Ack=2881 Win=63360 Len=0
12	0.285889358	80.249.99.148	10.0.2.15	TCP	4376	80 → 58796 [ACK] Seq=2881 Ack=105 Win=65535 Len=4320 [TCP seg…



Apply a display filter … <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
871	2.791169162	10.0.2.15	80.249.99.148	TCP	56	58796 → 80 [ACK] Seq=105 Ack=10444621 Win=65535 Len=0
872	2.791337061	80.249.99.148	10.0.2.15	TCP	1496	80 → 58796 [PSH, ACK] Seq=10444621 Ack=105 Win=65535 Len=1440…
873	2.791350976	80.249.99.148	10.0.2.15	TCP	8696	80 → 58796 [ACK] Seq=10446061 Ack=105 Win=65535 Len=8640 [TCP…
874	2.791399715	80.249.99.148	10.0.2.15	TCP	28856	80 → 58796 [ACK] Seq=10454701 Ack=105 Win=65535 Len=28800 [TC…
875	2.791519370	80.249.99.148	10.0.2.15	TCP	2276	80 → 58796 [PSH, ACK] Seq=10483501 Ack=105 Win=65535 Len=2220…
876	2.792097345	10.0.2.15	80.249.99.148	TCP	56	58796 → 80 [ACK] Seq=105 Ack=10485721 Win=65535 Len=0
877	2.810246472	80.249.99.148	10.0.2.15	HTTP	380	HTTP/1.1 200 OK (application/zip)
878	2.810282420	10.0.2.15	80.249.99.148	TCP	56	58796 → 80 [ACK] Seq=105 Ack=10486045 Win=65535 Len=0
879	2.811745716	10.0.2.15	80.249.99.148	TCP	56	58796 → 80 [FIN, ACK] Seq=105 Ack=10486045 Win=65535 Len=0
880	2.811920607	80.249.99.148	10.0.2.15	TCP	62	80 → 58796 [ACK] Seq=10486045 Ack=106 Win=65535 Len=0
881	2.950678819	80.249.99.148	10.0.2.15	TCP	62	80 → 58796 [FIN, ACK] Seq=10486045 Ack=106 Win=65535 Len=0
882	2.950701233	10.0.2.15	80.249.99.148	TCP	56	58796 → 80 [ACK] Seq=106 Ack=10486046 Win=63328 Len=0
883	5.070620362	PcsCompu_90:2b:13		ARP	44	Who has 10.0.2.3? Tell 10.0.2.15
884	5.071278091	52:55:0a:00:02:03		ARP	66	10.0.2.3 is at 52:55:0a:00:02:03

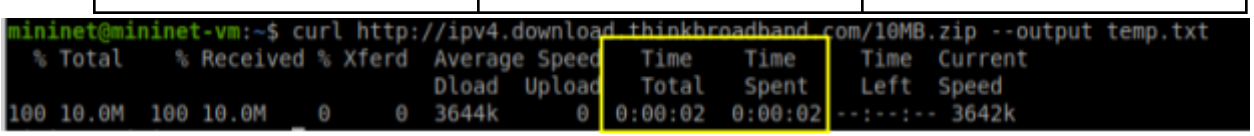
e. **Throughput according to Wireshark:** Using the result in (d), calculate the <u>average throughput</u> seen by the Client and enter it in the table below. Show all of your work.

The total throughput is 10 MB / 2.80502 sec = 3.565 MBytes/sec = 28.52 Mbps

f. Enter the throughput as reported in the curl terminal in the table below. How do the values in the table compare? Is it close? Discuss and account for any differences. Take a **<u>screenshot</u>** to show the reported download speed according to curl.

(Refer table below)

Application	Throughput	Notes (if any)
Wireshark capture	28.52 Mbps	
Terminal output	40 Mbps	10MB/2sec*8bits/Byte



```
mininet@mininet-vm:~$ curl http://ipv4.download.thinkbroadband.com/10MB.zip --output temp.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 10.0M  100 10.0M    0     0  3644k      0  0:00:02  0:00:02 --:--:-- 3642k
```

The throughput for the terminal output is much higher than the wireshark capture. This is due to the measurement differences. Wireshark provides the timing down to the msec. However, curl only reports the speed to the seconds precision. Furthermore, wireshark may have some more overhead because of all its packet capturing.